

2235. Add Two Integers

Solved

Easy Topics Companies

Given two integers num1 and num2, return the sum of the two integers.

Example 1:

Input: num1 = 12, num2 = 5

Output: 17

Explanation: num1 is 12, num2 is 5, and their sum is 12 + 5 = 17, so 17 is returned.

Example 2:

Input: num1 = -10, num2 = 4

Output: -6

Explanation: num1 + num2 = -6, so -6 is returned.

Constraints:

- 100 <= num1, num2 <= 100

GoDaddy Get email that looks as professional as your business is

Code

Java Auto

```
1 class Solution {
2     public int sum(int num1, int num2) {
3         return num1 + num2;
4     }
5 }
```

Saved

Ln 3, Col 27

Testcase Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

num1 = 12

num2 = 5

Output

258. Add Digits

Solved ✓

Easy Topics Companies Hint

Given an integer `num`, repeatedly add all its digits until the result has only one digit, and return it.

Example 1:

Input: `num = 38`

Output: `2`

Explanation: The process is

`38 --> 3 + 8 --> 11`

`11 --> 1 + 1 --> 2`

Since 2 has only one digit, return it.

Example 2:

Input: `num = 0`

Output: `0`

Constraints:

- $0 \leq \text{num} \leq 2^{31} - 1$

Code

Java Auto

```
1 class Solution {
2     public int addDigits(int num) {
3         while (num > 9)
4         {
5             int sum = 0;
6             while (num > 0)
7             {
8                 sum += num % 10;
9                 num /= 10;
10            }
11            num = sum;
12        }
13        return num;
14    }
15 }
16 }
```

Saved

Ln 13, Col 19

509. Fibonacci Number

Solved

Easy Topics Companies

The **Fibonacci numbers**, commonly denoted $F(n)$ form a sequence, called the **Fibonacci sequence**, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

$$F(0) = 0, F(1) = 1$$

$$F(n) = F(n - 1) + F(n - 2), \text{ for } n > 1.$$

Given n , calculate $F(n)$.

Example 1:

Input: $n = 2$

Output: 1

Explanation: $F(2) = F(1) + F(0) = 1 + 0 = 1$.

Example 2:

Input: $n = 3$

Output: 2

Explanation: $F(3) = F(2) + F(1) = 1 + 1 = 2$.

Example 3:

Input: $n = 4$

Output: 3

9.4K 184

79 Online

Code

Java Auto

```
1 class Solution {
2     public int fib(int n) {
3         if(n<=1)
4             {
5                 return n;
6             }
7         return fib(n-1)+ fib(n-2);
8     }
9 }
```

Saved

Ln 5, Col 22

Testcase Test Result

Problem List

00:48:57

Premium

Description

Accepted

Editorial

Solutions

Submissions

1295. Find Numbers with Even Number of Digits

Solved

EasyTopicsCompaniesHint

Given an array `nums` of integers, return how many of them contain an **even number** of digits.

Example 1:

Input: `nums = [12,345,2,6,7896]`
Output: 2
Explanation:
12 contains 2 digits (even number of digits).
345 contains 3 digits (odd number of digits).
2 contains 1 digit (odd number of digits).
6 contains 1 digit (odd number of digits).
7896 contains 4 digits (even number of digits).
Therefore only 12 and 7896 contain an even number of digits.

Example 2:

Input: `nums = [555,901,482,1771]`
Output: 1
Explanation:
Only 1771 contains an even number of digits.

Constraints:

3K210

11 Online

Code

JavaAuto

```
1 class Solution {
2     public int findNumbers(int[] nums) {
3         if(nums.length == 0)
4         {
5             return 0 ;
6         }
7         int count = 0;
8         for(int i= 0;i<nums.length;i++)
9         {
10             int numsOfDigits = 0;
11             while(nums[i]!=0)
12             {
13                 nums[i] = nums[i]/10;
14                 numsOfDigits++;
15             }
16             if(numsOfDigits%2 ==0)
17             {
18                 count++;
19             }
20         }
21         return count;
22     }
23 }
```

Saved

Ln 13, Col 24

TestcaseTest Result

2160. Minimum Sum of Four Digit Number After Splitting Digits Solved ✓

Easy Topics Companies Hint

You are given a **positive** integer `num` consisting of exactly four digits. Split `num` into two new integers `new1` and `new2` by using the **digits** found in `num`. **Leading zeros** are allowed in `new1` and `new2`, and **all** the digits found in `num` must be used.

- For example, given `num = 2932`, you have the following digits: two 2's, one 9 and one 3. Some of the possible pairs `[new1, new2]` are `[22, 93]`, `[23, 92]`, `[223, 9]` and `[2, 329]`.

Return the **minimum** possible sum of `new1` and `new2`.

Example 1:

Input: `num = 2932`

Output: 52

Explanation: Some possible pairs `[new1, new2]` are `[29, 23]`, `[223, 9]`, etc. The minimum sum can be obtained by the pair `[29, 23]`: $29 + 23 = 52$.

Example 2:

Input: `num = 4009`

Output: 13

Explanation: Some possible pairs `[new1, new2]` are `[0, 49]`, `[490, 0]`, etc. The minimum sum can be obtained by the pair `[4, 9]`: $4 + 9 = 13$.

👍 1.5K 🗨️ 49 ☆ 📌 ⓘ

8 Online

Code

Java 🔒 Auto

```
1 class Solution
2 {
3     public int minimumSum(int num)
4     {
5         int [] digits = new int [4];
6         for(int i = 3; i >=0;i--)
7         {
8             digits[i] = num%10;
9             num = num/10;
10        }
11        Arrays.sort(digits);
12        int num1 = digits[0]*10 + digits[2];
13        int num2 = digits[1]*10 + digits[3];
14        return num1 + num2 ;
15    }
16 }
```

Saved

Ln 13, Col 41

✓ Testcase > Test Result

2520. Count the Digits That Divide a Number

Solved

Easy Topics Companies Hint

Given an integer `num`, return the number of digits in `num` that divide `num`.

An integer `val` divides `nums` if `nums % val == 0`.

Example 1:

Input: `num = 7`

Output: 1

Explanation: 7 divides itself, hence the answer is 1.

Example 2:

Input: `num = 121`

Output: 2

Explanation: 121 is divisible by 1, but not 2. Since 1 occurs twice as a digit, we return 2.

Example 3:

Input: `num = 1248`

Output: 4

Explanation: 1248 is divisible by all of its digits, hence the answer is 4.

Code

Java Auto

```
1 class Solution {
2     public int countDigits(int num) {
3         int temp = num;
4         int count = 0;
5         while (temp != 0) {
6             int digit = temp % 10;
7             if (num % digit == 0) {
8                 count++;
9             }
10            temp /= 10;
11        }
12        return count;
13    }
14 }
```

Saved

Ln 12, Col 22

1512. Number of Good Pairs

Solved

Easy Topics Companies Hint

Given an array of integers `nums`, return the number of **good pairs**.

A pair (i, j) is called **good** if `nums[i] == nums[j]` and $i < j$.

Example 1:

Input: `nums = [1,2,3,1,1,3]`

Output: 4

Explanation: There are 4 good pairs $(0,3)$, $(0,4)$, $(3,4)$, $(2,5)$ 0-indexed.

Example 2:

Input: `nums = [1,1,1,1]`

Output: 6

Explanation: Each pair in the array are **good**.

Example 3:

Input: `nums = [1,2,3]`

Output: 0

Constraints:

5.9K 142

15 Online

Code

Java Auto

```
1 class Solution {
2     public int numIdenticalPairs(int[] nums) {
3         int count = 0;
4         int n = nums.length;
5         for(int i = 0; i < n; i++)
6         {
7             for(int j = i + 1; j < n; j++)
8             {
9                 if(nums[i] == nums[j])
10                {
11                    count++;
12                }
13            }
14        }
15        return count;
16    }
17 }
18 }
```

Saved

Ln 15, Col 22

Testcase Test Result

1929. Concatenation of Array

Solved ✓

Easy Topics Companies Hint

Given an integer array `nums` of length `n`, you want to create an array `ans` of length `2n` where `ans[i] == nums[i]` and `ans[i + n] == nums[i]` for $0 \leq i < n$ (0-indexed).

Specifically, `ans` is the **concatenation** of two `nums` arrays.

Return the array `ans`.

Example 1:

Input: `nums = [1,2,1]`

Output: `[1,2,1,1,2,1]`

Explanation: The array `ans` is formed as follows:

- `ans = [nums[0],nums[1],nums[2],nums[0],nums[1],nums[2]]`
- `ans = [1,2,1,1,2,1]`

Example 2:

Input: `nums = [1,3,2,1]`

Output: `[1,3,2,1,1,3,2,1]`

Explanation: The array `ans` is formed as follows:

- `ans = [nums[0],nums[1],nums[2],nums[3],nums[0],nums[1],nums[2],nums[3]]`
- `ans = [1,3,2,1,1,3,2,1]`

Code

Java Auto

```
1 class Solution {
2     public int[] getConcatenation(int[] nums) {
3         int ans[] = new int[2*nums.length];
4         for (int i=0;i<nums.length;i++)
5         {
6             ans[i]=nums[i];
7         }
8         int index = nums.length;
9         for(int i =0;i<nums.length;i++)
10        {
11            ans[index]=nums[i];
12            index++;
13        }
14        return ans;
15    }
16 }
```

Saved

Ln 14, Col 20

1323. Maximum 69 Number

Solved ✓

Easy Topics Companies Hint

You are given a positive integer `num` consisting only of digits `6` and `9`.

Return the maximum number you can get by changing **at most** one digit (`6` becomes `9`, and `9` becomes `6`).

Example 1:

Input: `num = 9669`

Output: `9969`

Explanation:

Changing the first digit results in `6669`.

Changing the second digit results in `9969`.

Changing the third digit results in `9699`.

Changing the fourth digit results in `9666`.

The maximum number is `9969`.

Example 2:

Input: `num = 9996`

Output: `9999`

Explanation: Changing the last digit `6` to `9` results in the maximum number.

Example 3:

Input: `num = 9999`

👍 3.3K 🗨️ 428 ☆ 📌 ⓘ

17 Online

Code

Java 🔒 Auto

```
1 class Solution {
2     public int maximum69Number (int num) {
3         int div = 1;
4         while (num / div >= 10) {
5             div *= 10;
6         }
7         while (div > 0) {
8             int digit = (num / div) % 10;
9
10            if (digit == 6) {
11                num = num + 3 * div;
12                break;
13            }
14            div /= 10;
15        }
16        return num;
17    }
18 }
```

Saved

Ln 17, Col 6

✅ Testcase ➤ Test Result

Description Accepted × Editorial Solutions Submissions

2469. Convert the Temperature

Solved ✓

Easy Topics Companies Hint

You are given a non-negative floating point number rounded to two decimal places `celsius`, that denotes the temperature in Celsius.

You should convert Celsius into **Kelvin** and **Fahrenheit** and return it as an array `ans = [kelvin, fahrenheit]`.

Return the array `ans`. Answers within 10^{-5} of the actual answer will be accepted.

Note that:

- `Kelvin = Celsius + 273.15`
- `Fahrenheit = Celsius * 1.80 + 32.00`

Example 1:

Input: `celsius = 36.50`

Output: `[309.65000, 97.70000]`

Explanation: Temperature at 36.50 Celsius converted in Kelvin is 309.65 and converted in Fahrenheit is 97.70.

Example 2:

Input: `celsius = 122.11`

Output: `[395.26000, 251.79800]`

Code

Java Auto

```
1 class Solution {
2     public double[] convertTemperature(double celsius) {
3         double kelvin = celsius + 273.15;
4         double Fahrenheit = celsius * 1.80 + 32.00;
5         return new double [] {kelvin, Fahrenheit};
6     }
7 }
```

Saved

Ln 5, Col 51

764 104 ☆ ↗ ?

9 Online

Testcase Test Result